Selecting the archive's name-search instrument: experiment design

Experiment design · 10 October 2026 · evaluation data: `extracted_text_old.csv`

SUMMARY

The experiment selects the search instrument that the archive will operate by comparing candidate generators (cross-script embedding models and lexical phonetic baselines), equivalence dictionaries and ranking functions on queries derived from the transcribed records. A sequential design replaces a full factorial: each of three development experiments varies one component, holds the others at the previous experiment's best setting and carries the two strongest configurations forward, and a final confirmation run measures the frozen pipeline once on a held-out test split. A profile of the evaluation data shaped the design. Only 38 % of the 473 Spanish–Hebrew given-name pairs on the stones are linked by any phonetic method or by the embedding encoder; most of the remainder share only an initial sound, the pattern of civil names chosen to echo a Hebrew or Yiddish name. Equivalence dictionaries are therefore a primary instrument for given names, and one of them is learned from the archive itself under fold-wise separation.

1 Research question

Which search instrument, defined as a candidate generator, an equivalence dictionary and a ranking function, retrieves the intended person most reliably for name queries typed in Latin or Hebrew script, within the latency and memory limits of the Flask service? Three sub-questions follow the components, and each is answered by one development experiment.

- **RQ1 (generators).** Which embedding model, and which lexical phonetic method, retrieves the intended person best without any dictionary?
- **RQ2 (dictionaries).** How much does an equivalence dictionary add, which dictionary source performs best (curated, archive-derived or both), and where in the pipeline should the dictionary act?
- **RQ3 (ranking).** Which ranking function orders the candidates best, and does fusing several generators improve on the best single one?

The primary measure is recall at 10 (R@10), the share of queries whose intended person appears on the first page of results. Mean reciprocal rank (MRR), the number of unrelated people among the first ten results shown above the instrument's display cut-off, and the 95th-percentile query latency are secondary measures. The selected instrument is the simplest configuration whose R@10 lies within the 95 % confidence interval of the best configuration and whose latency stays under a budget of 500 ms, fixed before the runs begin; simplicity is counted in components.

2 Evaluation data

The extraction file holds 828 person rows from 794 graves (628 in the women's section, 200 in the children's). Of these, 668 rows record a given name and a surname in at least one script: 587 in Latin script, 444 in Hebrew script and 366 in both. Three properties of the records bear directly on the design.

- **Given names in the two scripts are often different names.** Figure 1 classifies the 473 rows that carry a given name in both scripts. In 182 pairs (38 %) the two forms are linked by Daitch–Mokotoff, Beider–Morse or the byte-level encoder (Rebeca–רבקה, Samuel–שמואל). In 191 pairs (40 %) the forms share only their initial consonant class, which covers cognates the methods miss (José–יוסף, Raquel–רחל) and civil names chosen to echo the Hebrew or Yiddish name (Pascual–פסח, Marcos–מרדכי, Cecilia–צירל). The remaining 100 pairs (21 %) share neither, as with translations such as Adolfo or Valentín for זאב ('wolf').
- **Married women carry composite surnames.** Of 513 Latin-script surnames in the women's section, 411 contain the married-name particle 'de' ('S. de Blejer', 'de Frenkel'), and the Hebrew surname on the same stone may record the maiden name, the married name or both. Surnames must be tokenised, the particle and initials removed, and every surname token indexed.

- **Hebrew script is largely Yiddish orthography.** 188 of the 448 Hebrew-script surnames contain ⟨ע⟩ or ⟨טש⟩, the Yiddish spellings of /e/ and /tʃ/ (סערעברינסקי, טאלקאטשיער), which neither Modern Hebrew romanisation nor the embedding models' training data represent well, and 127 of 583 Hebrew given names are double names (חיה שרה).

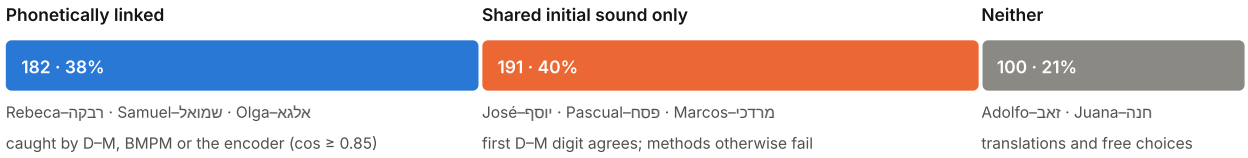


Figure 1. Relation between the Spanish and the Hebrew given name recorded for the same person (473 rows), classified automatically. A pair counts as phonetically linked when any token of the Hebrew name matches the first Spanish token under Daitch–Mokotoff after AnyAscii romanisation, under Beider–Morse with its Hebrew-value pass, or at encoder cosine ≥ 0.85; the classification is approximate and serves to size the problem.

2.1 Preparation, splits and query sets

The evaluation set comprises the 657 eligible rows with extraction confidence of at least 85. Rows that describe the same person on two stones (the same names and year of death, as with the Serebrinsky brothers on stones 7 and 8 of the children's section) are merged into one target with two records, both counted as correct. Names are normalised exactly as the production import will normalise them: case, accents, Hebrew points and final letter forms, the particle 'de' and initials. Twenty per cent of persons, stratified by section and by script coverage, form a **test split** that is touched only in Experiment 4; the other eighty per cent form a **development split** evaluated by five-fold cross-validation, with all graves of one family surname kept in the same fold so that no surname appears in both a dictionary's training folds and its evaluation fold.

Queries are generated from the records in five sets (Table 1). The cross-script sets use **field masking**: for a query typed in one script, the target person's fields in that script are removed from the index, so that the person can be found only through the other script, which is the situation of the 221 persons recorded in Latin script alone and the 78 recorded in Hebrew script alone.

Set	Query	Construction	Eligible persons
QS1	Latin name with typing errors	Recorded Latin given name and surname, one or two random edits (deletion, transposition, adjacent-key substitution), in either order	587
QS2	Diaspora spelling	A different Latin spelling of the same name as a descendant abroad might type it (Kofman → Kaufman, Hurvitz → Horowitz), produced by transliteration rules and an LLM and checked by hand on a sample	587
QS3	Hebrew → Latin record	Recorded Hebrew given name and surname; the target's Hebrew fields masked	366
QS4	Latin → Hebrew record	Recorded Latin given name and surname; the target's Latin fields masked	366
QS5	Partial name	Surname alone, or given name alone, drawn from QS1–QS4 (for women, the husband's surname where recorded)	sample of 300

Table 1. Query sets. Results are reported per set as well as pooled, since the sets test different abilities: QS1 and QS2 within-script variation, QS3 and QS4 cross-script correspondence and name equivalence, and QS5 the many-candidate regime of a single common name.

3 Instruments

An instrument is assembled from one entry in each block of Table 2. Lexical baselines are included alongside the embedding models because the literature review found romanise-then-compare pipelines competitive with learned encoders [1], and a claim that one instrument is optimal requires comparison with the cheapest credible alternative.

ID	Instrument	Runs	Notes
Embedding models (candidate generators)			
E1	Byte-level contrastive encoder [2]	Local, CPU	4M parameters, trained on 4.67M LLM-generated name pairs across eight scripts including Hebrew; runs on CPU in NumPy
E2	Symphonym [1]	Local, CPU	8.3M-parameter character model distilled from articulatory features; twenty scripts including Hebrew; trained on toponyms

ID	Instrument	Runs	Notes
E3a	LaBSE [3]	Local, CPU	General-purpose multilingual sentence encoder; control for the semantic-embedding route discussed in September
E3b	multilingual-e5-small [8]	Local, CPU	Smaller general-purpose multilingual encoder, a second local control
E4	E1 fine-tuned on archive pairs (optional)	Local, CPU	Contrastive fine-tuning on the Spanish–Hebrew name pairs of the training folds only, to adapt the encoder to Yiddish orthography
Lexical baselines (candidate generators)			
L1	Exact match after normalisation	PostgreSQL	Floor for every metric
L2	Trigram similarity (`pg_trgm`)	PostgreSQL	Already in the data-tier design; within-script only
L3	Daitch–Mokotoff on Latin names and on AnyAscii-romanised Hebrew	PostgreSQL	Current design; native PostgreSQL
L4	Beider–Morse with the Hebrew-value pass, Ashkenazic rules, fixed language set	Python + PostgreSQL	Abydos in Python; codes stored in an indexed array column
Equivalence dictionaries			
D0	None	—	Control
D1	Curated given-name table	PostgreSQL table	About one hundred groups (Hebrew, Yiddish, Spanish forms) seeded from the JewishGen Given Names Data Bases for Argentina [4], compiled without reference to the test split
D2	Archive-derived table	PostgreSQL table	Spanish–Hebrew given-name and surname pairs that co-occur on the same person in the training folds, kept at a minimum support of two; rebuilt for every fold
D3	D1 ∪ D2	PostgreSQL table	—
D4	Initial-sound rule (optional)	Python	A weak equivalence between names whose romanised initial consonant class agrees, motivated by Figure 1; used only as a ranking feature
Ranking functions			
K1	Maximum similarity	Flask (Python)	Score of the best-matching name field
K2	Route-weighted score	Flask (Python)	Direct matches by similarity; dictionary-derived matches by the similarity of their seed times a penalty λ
K3	Reciprocal rank fusion [5]	Flask (Python)	Sums 1/(*k* + rank) across the lists of several generators; needs no score calibration
K4	Learned linear scorer	Flask (Python)	Logistic regression over cosine, trigram similarity, D–M and BMPM agreement, route, initial-sound agreement, field type and name length, fitted on the training folds

Table 2. Instruments under test. Every component runs inside the project's own Flask service or PostgreSQL database. Every ranking function scores people, combining a given-name component and a surname component when the query contains both, so that a full-name query ranks a person highly only when both parts match.

4 Methods

Figure 2 shows the sequence. A preparation stage builds the evaluation set once; Experiments 1–3 then run on the development split; each varies one component, fixes the others at the previous experiment's best setting and passes its two strongest configurations forward, which bounds the number of runs at about twenty, against several hundred for the full factorial, while keeping a second candidate alive wherever two components might interact. Thresholds, *k*, λ and the K4 weights are tuned inside the training folds only.

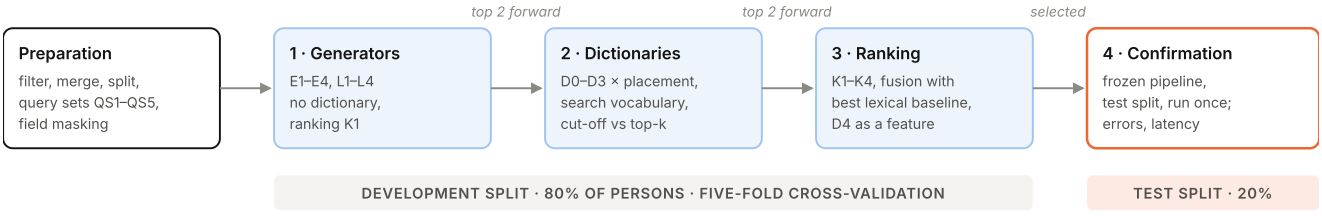
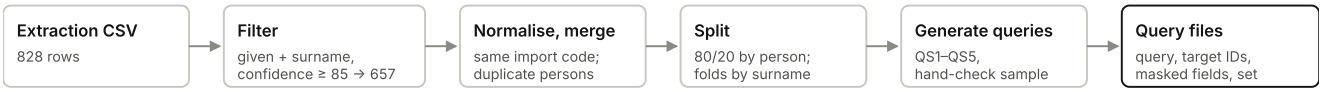


Figure 2. Sequential design. Each development experiment varies one component and carries its two best configurations forward; the test split is used once, by the pipeline frozen at the end of Experiment 3.

Preparation · Building the evaluation set



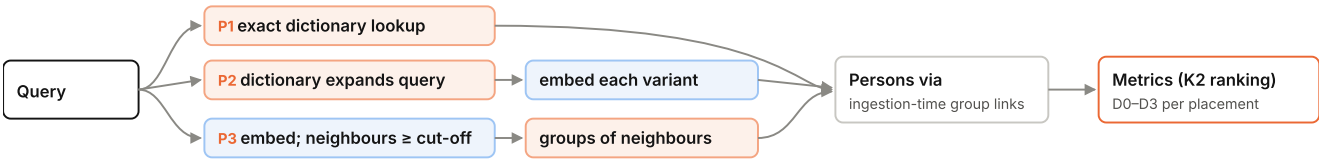
- Apply the eligibility filter and the import's own normalisation, so that the experiment tests the code path that will run in production.
- Merge duplicate persons into single targets and record every grave as a correct answer.
- Draw the test split (20 %, stratified by section and script coverage) and the five development folds, grouping graves by family surname.
- Generate QS1–QS5 with a fixed random seed; hand-check 50 QS2 variants for plausibility, and generate QS2 with a model other than the one that produced E1's training pairs, to avoid favouring E1.
- Report results separately on the rows that fall in the hand-checked transcription sample, where the target names are known to be correct.

Experiment 1 · Candidate generators (RQ1)



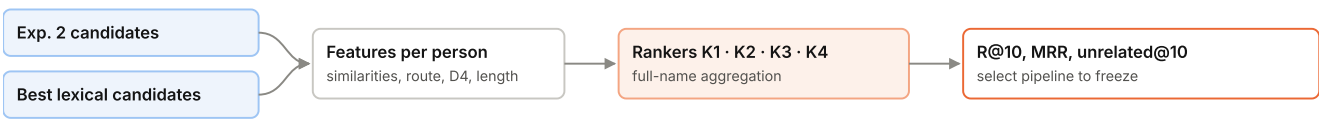
- Index every name field of every development person with each generator, one generator at a time, with no dictionary and maximum-similarity ranking.
- For embeddings, retrieve with a cut-off and a top-k cap; tune the cut-off on the training folds for R@10 under an unrelated@10 limit.
- Compare generators on R@10 per query set with paired bootstrap confidence intervals [6]; carry forward the best two embedding models and the best lexical baseline.
- If E4 is run, fine-tune within each fold on that fold's training pairs only.

Experiment 2 · Dictionary source and placement (RQ2)



- Fix the generators from Experiment 1 and compare three placements of the dictionary: dictionary only (P1), dictionary before the embedding (P2), and embedding before the dictionary (P3, the pipeline proposed on 10 October), each with D1, D2 and D3 against D0.
- For P3, test the two settings that moved results most in the October simulation: searching the archive's spellings alone against the archive's spellings plus the dictionary's, and a similarity cut-off against a fixed top-k.
- Link every archive spelling to its dictionary group at import (exact entry, else nearest dictionary spelling above a cut-off); for D2 this happens per fold. Record the share of links a reviewer would reject on a sample of 100.
- Report QS3 and QS4 separately for given names and surnames, since the dictionaries matter mainly for given names (Figure 1), and carry forward the best two configurations.

Experiment 3 · Ranking and fusion (RQ3)



- Pool the candidates of the Experiment 2 configurations with those of the best lexical baseline and score each person with K1–K4.
- Aggregate the given-name and surname components per person and compare a sum with a product, which penalises a strong match on one part alone.
- Fit λ (K2) and the K4 weights on the training folds; inspect the K4 coefficients to see which signals carry weight, including the D4 initial-sound feature.
- Select the pipeline to freeze by the decision rule of §1 and record its full configuration before opening the test split.

Experiment 4 · Confirmation on the test split

- Run the frozen pipeline once on the test queries, with dictionaries D2 rebuilt from the whole development split, and report every metric with 95 % bootstrap intervals, alongside the current design (L3 with K1) as the reference.
- Compare the selected pipeline with the runner-up on R@10 with McNemar's test for paired outcomes [7].
- Break results down by query set, section, script direction, name length and extraction confidence, and examine every QS3–QS4 failure by hand, sorting each into a missing equivalence, a transcription error or a ranking error.
- Measure latency on CPU at the Render instance size (cold and warm), including query embedding and the PostgreSQL lookups.

5 Threats to validity

The targets come from automatic extraction, so some intended names are themselves misread (שמרן for שמואל on one Serebrinsky stone); the hand-checked subset bounds this effect. QS2 relies on generated spellings, which may resemble E1's LLM-generated training data, and the hand check and the use of a different generating model mitigate but do not remove that bias. The archive-derived dictionary is learned from the evaluation data, and fold-wise construction is what keeps its gains honest. With about 130 test persons the intervals will be wide, and differences of a few points of R@10 should be read as ties. The file evaluated is the older extraction; the protocol should be rerun unchanged on the final transcription before deployment.

References

[1] Gadd, S. (2026). Symphonym: Universal phonetic embeddings for cross-script name matching. arXiv:2601.06932.

[2] Jumle, V., et al. (2026). Cross-script phonetic name retrieval via byte-level contrastive embeddings. TU Delft DSAIT4050 project. https://github.com/vedant-jumle/cross-language-phonetic-text-alignment

[3] Feng, F., Yang, Y., Cer, D., Arivazhagan, N., & Wang, W. (2022). Language-agnostic BERT sentence embedding. *Proc. ACL 2022*, 878–891.

[4] Esterson, G. L., & Curwin, D. Given Names Data Bases. JewishGen. https://www.jewishgen.org/databases/givennames/

[5] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *Proc. SIGIR '09*, 758–759.

[6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

[7] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.

[8] Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). Multilingual E5 text embeddings: A technical report. arXiv:2402.05672.

Figures in §2 were computed from `extracted_text_old.csv` with the scripts accompanying this PDF; the companion literature review documents the methods behind E1, E2, L2–L4 and the October pipeline simulation.